

Hello Everyone!

We are back!

1

Class Overview

In Class:

- Lifecycle-Based Testing
- Model-Based Testing
- Mutation Testing
- Reserved Time: Mutation Testing Software Help

After Class Office Hours: Mutation Testing Software Help

Homework:

- 04-01: Mutation Testing

2

CIS 613: Lifecycle-Based Testing

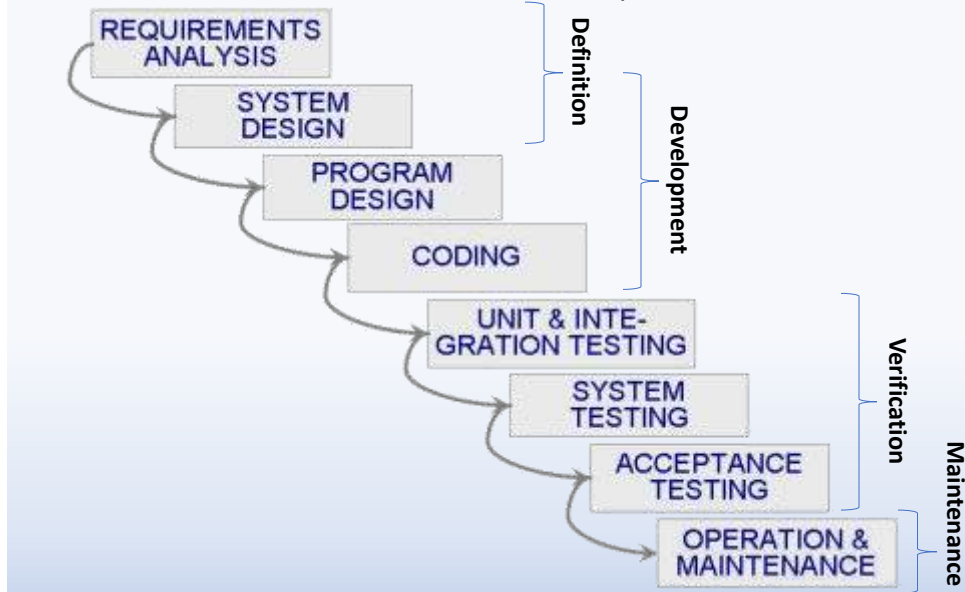
3

Outline

- Waterfall (& V-Model)
- Iterative
- Rapid Prototyping
- Executable Specification
- Agile Methods

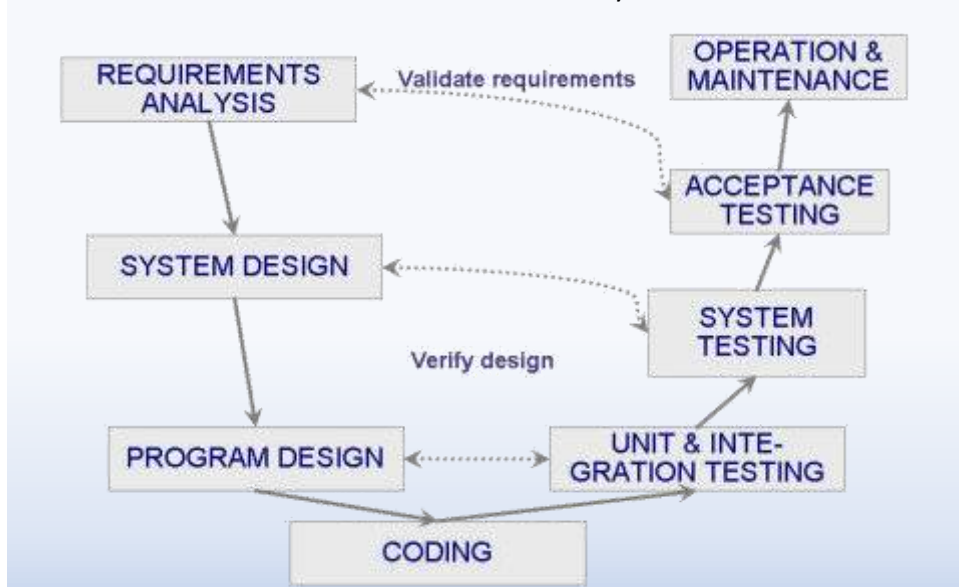
4

The Waterfall Lifecycle



5

The V-Model Lifecycle



6

Evaluation of the Waterfall Model

Advantages

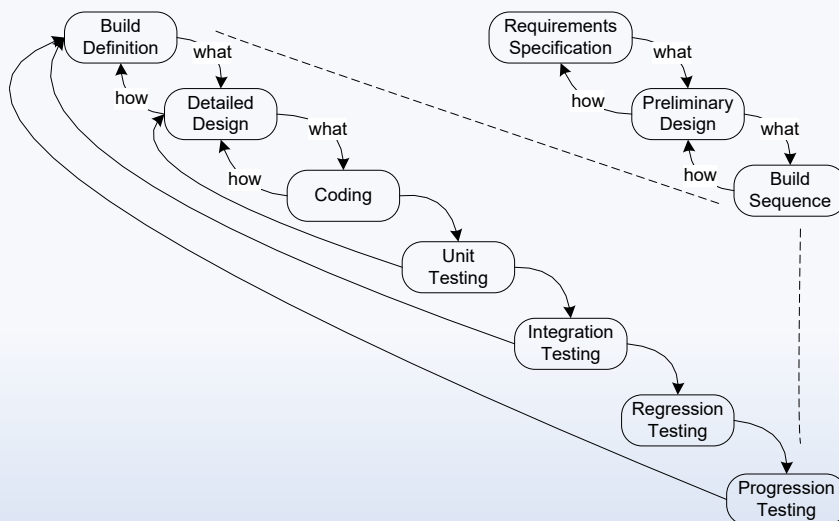
- Hierarchical structure maps nicely into large projects
- Phases have well-defined end products
- Unit level work can be done in parallel, reducing overall project interval

Disadvantages

- Extremely long feedback cycle for customer
- Very late synthesis (begins at integration testing)
- Staff limitations may not support the advantage of massive parallel development at the unit level
- Requires “perfect foresight”, otherwise early faults propagate

7

Iterative Development



8

Iterative Development

- Preserves a single high level design phase
 - Amortizing design across increments is risky. Early design decisions may eliminate later design choices
 - Defines the sequence and content of “builds” (or increments)
- Builds create the need for regression testing
- Preserves the advantages of Waterfall, AND
- **Responds to Waterfall defects**
 - Staffing limitations
 - Late synthesis
 - Long feedback cycle with customer

9

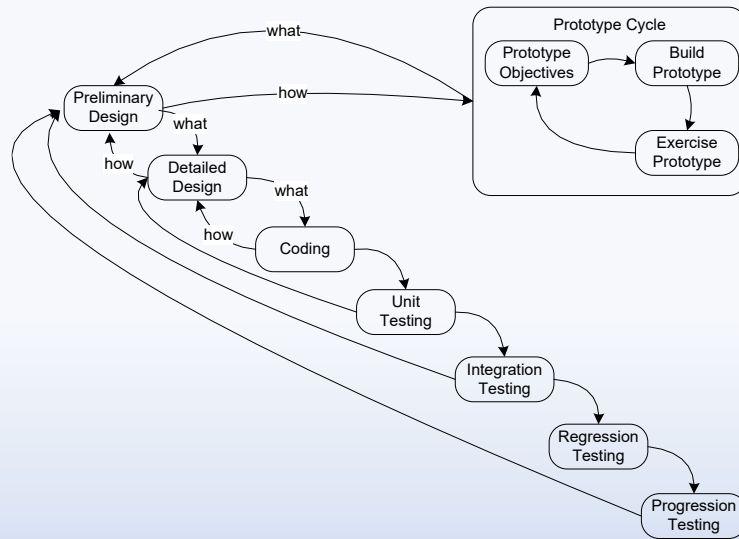
Waterfall & Iterative “Perfect Foresight?”

- Waterfall and the iterative variations have no answer for the customer who does not have a clear, complete idea of what is needed.
- “Requirements Elicitation” is the process of helping customers and developers reach a common understanding of a proposed system.

How does the assumption of perfect
foresight impact testing?

10

Rapid Prototyping



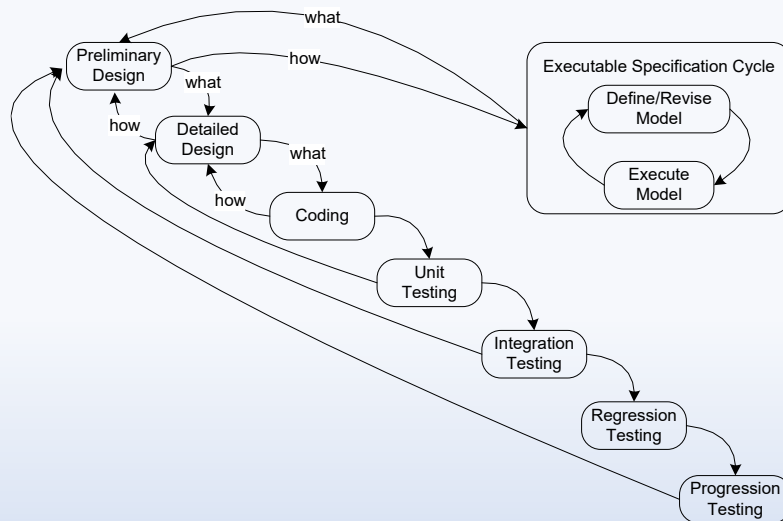
11

Rapid Prototyping

- Helps customer identify needs and defects
 - “I’ll know what I want when I see it.”
 - provides the “does view” that customers appreciate
 - ideal to give the “look and feel” of menu-driven systems
 - modify prototype per customer feedback
 - Sometimes done for feasibility
- **Advantages:**
 - improved and early feedback with customer
 - better basis for design
- Keep or dispose?
 - once it has served its purpose, the prototype can be archived.
 - possible to use to identify test scenarios

12

Executable Specification



13

Evaluation of Executable Specification

- Intended for “reactive systems” (event-driven)

- **Advantages**

- early feedback
- automatic generation of system test cases
- can be used for operator training
- support for early analysis

- **Disadvantages**

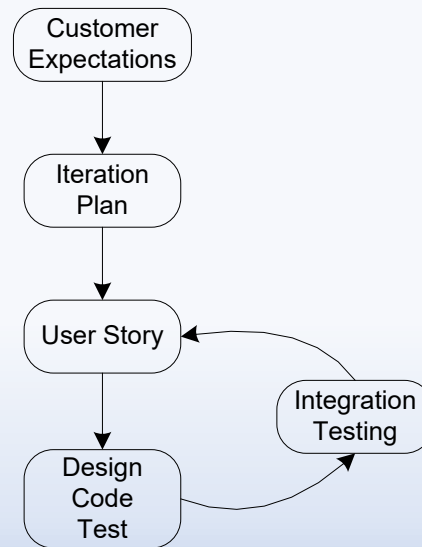
- modeling can be difficult
- training may be necessary
- engine can be expensive

14

Responses to Previous Methods

General Agile Lifecycles:

- Extreme Programming
- Test Driven Development
- Scrum
- Response to lack of “perfect foresight”
- Customer-driven
- *Short* increments



15

Test-Driven Development

- Extreme case of agile development
- Bottom-up development based on test cases
 - derived from customer-provided user stories
 - very quick feedback
- Very small increments
 - early synthesis
 - excellent fault isolation
 - refactoring results in clean code
- BUT, no opportunity for a comprehensive design

16

Advantages of Test-Driven Development

- The steps are deliberately small.
- Customer and developer can (should!) jointly determine granularity of user stories.
- Fault isolation is greatly simplified (in fact, trivial). If a test case fails, the fault must be in the most recently added code.
- Once a new test case passes, a working (subset) of the desired software can always be delivered.
- Something always works!

17

Disadvantages of Test-Driven Development

- Useful granularity is an issue.
- There is no guarantee that user stories “arrive” in a sensible order.
- There is no guarantee that user stories are the “same size” (or require similar effort)
- Bottom-up coding often results in poorly structured code, making refactoring necessary.
- Automated testing support (e.g. JUnit) is essential.

18

Test-Driven Development vs Model-Driven Development

Model-Driven Development is a rigorous, top-down approach.



Test-Driven Development is a bottom-up approach.